

# Assignment 4

## Adding cd to your Shell

Author: Mike Izbicki

This project must be done in a group of two

## Coding Requirements

You will make three modifications to your rshell project.

1. Implement the `cd` command. Every time this command is run, you must update the `PWD` and `OLDPWD` environment variables appropriately. This will require using the `chdir`, `getenv`, and `setenv` functions. Your implementation must support the following forms:
  - a. `cd <PATH>` will change the current working directory to `<PATH>`
  - b. `cd` will change the current working directory to the user's home directory
  - c. `cd -` will change the current working directory to the previous working directory
2. Change your prompt so that it displays the current working directory.

## Submission Instructions

Add this new code to your rshell project from assignment 3. Create a branch called `assn4`. Do all of your work under this branch. When finished, merge the test branch into the master branch, and create a tag called `hw4`. Remember that tags and branches in git are case sensitive!

NOTE: `git push` will not automatically push tags to your repository. Use `git push origin hw4` to update your repository to include the `hw4` tag.

To download and grade your homework, the TA will run the following commands from the hammer server:

```
$ git clone https://github.com/yourusername/rshell.git
$ cd rshell
$ git checkout hw4
$ make
$ bin/rshell
```

You should ssh into hammer.cs.ucr.edu and run the above commands to verify that you've submitted your code successfully. If you forget how to use git, two students from previous cs100 courses (Rashid Goshtasbi and Kyler Rynear) [made video tutorials](#) on the git commands needed to submit your assignments via Github.

**Do not wait to upload your assignment to Github until the project due date.** You should be committing and uploading your assignment continuously. If you wait until the last day and can't figure out how to use git properly, then you will get a zero on the assignment. NO EXCEPTIONS.

You will also need to create a file for submitting your partner and github information. Create a text file using vim with the following information: you and your partner's name, you and your partner's net IDs, and the github url of your assignment's repository.

**Follow this format EXACTLY:**

```
name1=
ucrnetid1=
name2=
ucrnetid2=
repourl=
```

Save the file as hw3 (**WITH NO EXTENSION**) and submit it to iLearn's to Assignment 4 submission link.

**Here's an example file:**

```
name1=Busra Celikkaya
ucrnetid1=bceli001
name2=Amirali Darvishzadeh
ucrnetid2=adar001
repourl=https://www.github.com/busrac/rshell.git
```

Your repository should be public, however if you prefer to have a private repository please email the instructor for additional information on adding collaborators.

## Project Structure

There are no changes to your project structure. You simply need to add a `cd_test.sh` file which tests your various cd commands.

# Coding Conventions

Your code must not generate any warnings on compilation.

You must follow the [CalTech coding guidelines](#), as stated in the syllabus.

Your final executable must have no memory leaks.

# Testing

Proper testing is the most important part of developing software. It therefore will have a very large impact on your grade.

It is not enough to simply show that your program works in some cases. You must show that it works in every possible edge case. **That means you must try to write test cases that will break your program!**

If you are unsure if your test cases are sufficient, ask one of the instructors to review them before the deadline.

# Collaboration Policy

You MAY NOT look at the source code of any other student.

You MAY discuss with other students in general terms how to use the unix functions.

You are ENCOURAGED to talk with other students about test cases. You are allowed to freely share ideas in this regard.

You are ENCOURAGED to look at [bash's source code](#) for inspiration.

# Grading

- 5 points for well commented code
- 5 points for following style guidelines
- 20 points for sufficient test cases
- 25 points for implementing cd PATH
- 25 points for implementing cd
- 20 points for implementing cd -

**IMPORTANT:** Your project structure is not explicitly listed in the grading schedule above, however not following the correct structure will result in a 20 point deduction.

**IMPORTANT:** Projects that do not correctly compile as described above will receive no points.